

CKA Intensive

Day 2 — Services, DNS, Ingress, Gateway API, NetworkPolicy

Target: Kubernetes v1.32

Today

1. **Morning quiz** (10 min, oral)
2. Services in depth (5 types, Endpoints, EndpointSlices, kube-proxy modes)
3. **Lab 3** — services & DNS
4. CoreDNS, search domains, NodeLocal DNSCache
5. Ingress — controller, resource, classes, TLS
6. **Gateway API expanded** — Gateway, HTTPRoute, attachment model
7. **Lab 3 (cont.) + Lab 3b** — Ingress + Gateway API
8. NetworkPolicy — ingress, egress, AND/OR semantics, default-deny
9. **Lab 4** — NetworkPolicy

Morning quiz format

10 minutes. 8–10 questions. Oral. You answer if called on; room debates if you're stuck.

Goal: hear yourself say "endpoints are empty because..." eight times before your real exam.

If you missed one yesterday — say so now. We backfill on the spot.

Why we have Services at all

Pods are mortal. Their IPs change. You can't write `curl 10.244.1.5` and expect it to work tomorrow.

A **Service** gives you:

- **Stable VIP** (ClusterIP) that doesn't change for the Service's lifetime
- **Stable DNS name** `<svc>.<ns>.svc.cluster.local`
- **Load balancing** across pods matched by `selector`
- An automatically maintained set of **Endpoints** (pod IPs + ports)

```
Pod IPs:    10.244.1.5, 10.244.2.7, 10.244.1.9    ← ephemeral
Service:    10.96.0.42                            ← stable for life
DNS:        web.lab3.svc.cluster.local            ← stable for life
```

Service types — all five

Type	Reachable from	Use case
ClusterIP (default)	inside cluster only	microservice → microservice
NodePort	every node IP : port (30000–32767)	dev, fronted by external LB, no-cloud setups
LoadBalancer	external IP from cloud LB	production external endpoint
ExternalName	DNS CNAME to external host	wrap a non-k8s service behind a stable cluster name
Headless (clusterIP: None)	DNS returns pod IPs directly	StatefulSets, client-side LB, custom routing

LoadBalancer = NodePort + a cloud LB. Without a cloud-controller, it stays **Pending** forever (on kind, install **cloud-provider-kind** or MetalLB).

ClusterIP

```
apiVersion: v1
kind: Service
metadata: { name: web }
spec:
  type: ClusterIP          # implicit default
  selector: { app: web }
  ports:
    - port: 80             # what the Service exposes
      targetPort: 8080     # what the pod listens on
      protocol: TCP        # also: UDP, SCTP
```

`port` is the Service's port; `targetPort` is the container's. If they're equal you can omit `targetPort`.

```
k expose deploy web --port=80 --target-port=8080
```

NodePort

```
spec:
  type: NodePort
  selector: { app: web }
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30080          # optional; otherwise random in 30000-32767
```

Reach it on **any node**: `curl <node-ip>:30080`.

kube-proxy listens on that port on every node and forwards to a backend pod (not necessarily on the same node — `externalTrafficPolicy: Local` keeps it local).

LoadBalancer

```
spec:
  type: LoadBalancer
  selector: { app: web }
  ports: [{ port: 80, targetPort: 8080 }]
```

What happens:

1. apiserver creates a NodePort under the hood
2. **cloud-controller-manager** sees the Service, calls the cloud API
3. Cloud LB is created with all nodes as targets on the NodePort
4. `status.loadBalancer.ingress[0].ip` (or hostname) is populated

On bare-metal / kind: install **MetalLB** (or `cloud-provider-kind`) to fulfill the role of cloud-controller for LB Services.

Headless Service

```
spec:
  clusterIP: None           # this makes it headless
  selector: { app: db }
  ports: [{ port: 5432 }]
```

DNS query returns **A records for every Ready pod IP** — no VIP, no kube-proxy load balancing. Client picks one (round-robin via DNS).

Required by **StatefulSets**: gives you `db-0.db.ns.svc.cluster.local`, `db-1.db.ns...`, ... per-pod stable DNS.

ExternalName

```
spec:  
  type: ExternalName  
  externalName: legacy-api.example.com
```

Pure DNS-level alias. A pod doing `curl http://legacy-api` (with search domains) ends up at `legacy-api.example.com`.

- No selector, no endpoints
- Implemented as a **CNAME** in CoreDNS
- Use for: migrating off external dependencies, hiding the real hostname behind a stable cluster name

Endpoints — the missing link

The **endpoints** of a Service are the actual pod IP:port targets it sends traffic to.

```
k get endpoints web
# NAME      ENDPOINTS                                     AGE
# web      10.244.1.5:8080,10.244.2.7:8080,10.244.1.9:8080 3m
```

If **endpoints** is **empty**:

1. Selector doesn't match any pods (label typo on Service or Pod)
2. Pods exist but are **not Ready** (failing readinessProbe)
3. No pods exist at all
4. Port name/number mismatch between Service **targetPort** and container port

The #1 debugging command for "my Service doesn't work."

EndpointSlices (the modern API)

EndpointSlices replace the legacy **Endpoints** object at scale.

- One Service with 5000 pods used to be one giant Endpoints object
- Now it's many smaller **EndpointSlice** objects (default chunk size: 100)
- Better watch performance, less etcd churn

```
k get endpointslices -l kubernetes.io/service-name=web  
k describe endpointslice <name>
```

The legacy **Endpoints** object still exists and is the one most kubectl plumbing shows you. Both are kept in sync.

kube-proxy modes

Mode	How	Best for
iptables (default)	Adds chains/rules per Service+endpoint	small/medium clusters
IPVS	In-kernel L4 LB with hash tables (O(1) lookup)	many services / high churn
nftables (GA in 1.32)	Modern kernel netfilter API	new clusters

Cilium can **replace kube-proxy entirely** with eBPF (`kube-proxy-replacement: strict`). Same Service semantics, no iptables/IPVS rules.

Check the current mode: `k -n kube-system logs ds/kube-proxy | head -20` or `k -n kube-system get cm kube-proxy -o yaml`.

externalTrafficPolicy

For NodePort/LoadBalancer Services:

```
spec:  
  externalTrafficPolicy: Local      # or Cluster (default)
```

- **Cluster** (default): traffic can hop to a pod on a different node (extra hop, source IP **lost**)
- **Local**: only routes to pods on the receiving node (preserves source IP; if no local pod, **drops** the connection)

Use **Local** when you need real client IPs (security audit, geo-IP). Pair with **healthCheckNodePort** so the LB stops sending traffic to nodes without a local pod.

Lab 3 — Services & DNS (part 1)

→ `trainees/day2/labs/lab3-services-ingress.md`

60 min total — part 1 here, part 2 after Ingress lecture.

Part 1 (45 min):

- ClusterIP + curl from a pod
- Headless service, verify per-pod A records
- Intentional selector mismatch — see empty endpoints
- Cross-namespace resolution

CoreDNS — what every cluster runs

Deployed in `kube-system` as a **Deployment** (2 replicas by default).

Pods get its **ClusterIP** via `/etc/resolv.conf`, set by kubelet:

```
nameserver 10.96.0.10
search prod.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

- `nameserver` = CoreDNS Service IP
- `search` = automatic suffixes — that's how `web` resolves to `web.prod.svc.cluster.local`
- `ndots:5` = "if the name has < 5 dots, try with the search suffixes first"

DNS resolution rules

From a pod in namespace `prod`:

Query	Resolves to
<code>web</code>	<code>web.prod.svc.cluster.local</code> <i>(adds current ns)</i>
<code>web.backend</code>	<code>web.backend.svc.cluster.local</code>
<code>web.backend.svc</code>	<code>web.backend.svc.cluster.local</code>
<code>web.backend.svc.cluster.local.</code>	itself (FQDN, trailing dot)
<code>google.com</code>	external resolver (CoreDNS forwards <code>forward . /etc/resolv.conf</code> upstream)

Note `ndots:5`: `web.backend` has 1 dot < 5, so CoreDNS tries search suffixes first (`web.backend.prod.svc.cluster.local` → NXDOMAIN → `web.backend.svc.cluster.local` → match). Use a trailing dot or fully qualified to skip search.

CoreDNS Corefile

```
.:53 {
  errors
  health { lameduck 5s }
  ready
  kubernetes cluster.local in-addr.arpa ip6.arpa {
    pods insecure
    fallthrough in-addr.arpa ip6.arpa
    ttl 30
  }
  prometheus :9153
  forward . /etc/resolv.conf {
    max_concurrent 1000
  }
  cache 30
  loop
  reload
  loadbalance
}
```

Stored in `kube-system/configmap/coredns`. Edit with `k -n kube-system edit cm coredns`; CoreDNS reloads on change.

Debug DNS from a pod

```
k run dbg --rm -it --image=nicolaka/netshoot -- sh
> nslookup web
> nslookup web.backend
> nslookup web.backend.svc.cluster.local
> dig +search +noall +answer web
> cat /etc/resolv.conf
```

Common pathologies:

- CoreDNS pod in `CrashLoopBackOff` (look at the Corefile — usually `loop` detected by the `loop` plugin)
- Upstream resolver dead (`forward .` target unreachable)
- ClusterIP changed (`kubectl get svc -n kube-system kube-dns`) but pod's `resolv.conf` cached the old one — restart pod

NodeLocal DNSCache (optional, often deployed)

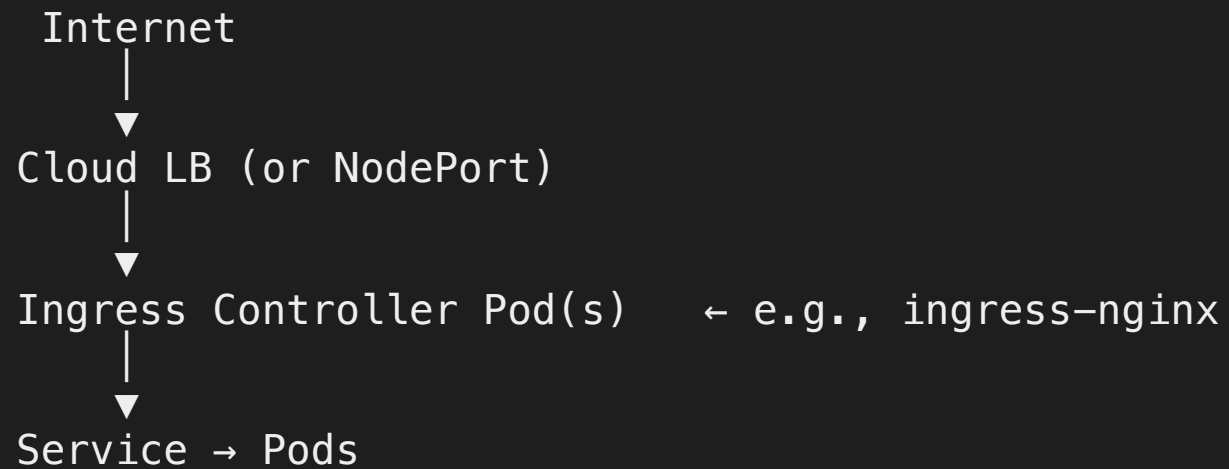
A DaemonSet pod on every node that caches DNS locally. Pods talk to `169.254.20.10` instead of CoreDNS Service IP.

Why: reduces CoreDNS load, eliminates `connt rack` table churn for UDP/53, lower tail latency.

Out of scope for the exam, but if you see `node-local-dns` pods in `kube-system` on a question's cluster, you'll know what it is.

Ingress — what it is

L7 (HTTP/HTTPS) routing into the cluster. Maps `host:path` → Service.



Crucial: an Ingress **resource** alone does nothing. You also need an **Ingress controller** running in the cluster. The controller watches Ingress objects and configures itself (its own nginx/envoy/haproxy/etc.) to route accordingly.

Ingress controllers (you choose)

- **ingress-nginx** — the reference; installed via Helm; most common
- **traefik** — popular for ease of setup
- **HAProxy Ingress**
- **AWS ALB / GCP Cloud LB / Azure App GW** — cloud-native, more flags but no controller pod to run
- **Contour, Kong, Emissary** — older niches

A cluster can run **multiple** controllers side-by-side. Use **IngressClass** to tell each Ingress which controller should handle it.

Ingress YAML

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx                # which controller
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /api
            pathType: Prefix              # Prefix | Exact | ImplementationSpecific
            backend:
              service:
                name: api
                port: { number: 80 }
          - path: /
            pathType: Prefix
            backend:
              service: { name: web, port: { number: 80 } }
  tls:
    - hosts: [app.example.com]
      secretName: app-tls                # kubernetes.io/tls Secret
```

pathType matters

Value	Match behavior
Exact	exact match of the URL path
Prefix	element-wise prefix match (/foo matches /foo, /foo/, /foo/bar ; does NOT match /foobar)
ImplementationSpecific	up to the controller (often regex-ish on ingress-nginx)

Almost always use Prefix. Older Ingresses without pathType are invalid since 1.18.

IngressClass

```
apiVersion: networking.k8s.io/v1
kind: IngressClass
metadata: { name: nginx }
spec:
  controller: k8s.io/ingress-nginx
```

You can mark one class as **default** with annotation `ingressclass.kubernetes.io/is-default-class: "true"` — Ingresses without `ingressClassName` then bind to it.

Multiple clusters running multiple controllers (nginx for public, AWS ALB for internal) — IngressClass is the dispatch.

Ingress TLS

```
spec:
  tls:
  - hosts: [app.example.com]
    secretName: app-tls
```

`app-tls` is a `kubernetes.io/tls` Secret:

```
k create secret tls app-tls --cert=tls.crt --key=tls.key
```

The controller terminates TLS at its edge. End-to-end TLS requires controller-specific config + serving HTTPS from the pod.

cert-manager (popular operator) automatically issues + renews certs from Let's Encrypt / private CAs.

Gateway API — the successor

The Ingress resource has problems:

- HTTP-only (no TCP/UDP/gRPC modeling)
- A swamp of vendor-specific annotations
- No traffic splitting, header rewriting, weighted routing in the spec
- Hard to split "who owns the listener" from "who owns the route"

Gateway API (GA in 1.31) is the formal replacement. CKA references it. Real clusters are adopting it now.

Gateway API — the three resources

```
GatewayClass — controller binding (Cluster admin owns)
  |
  ▼
  Gateway — listeners (ports, protocols, TLS) (Cluster ops own)
    |
    ▼ (attached via parentRefs)
    HTTPRoute / TCPRoute / GRPCRoute — hostnames + paths → backendRefs (App teams own)
```

Each layer is owned by a different role. That's *the* design intent: separate the network admin from the app team.

GatewayClass

```
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata: { name: contour }
spec:
  controllerName: projectcontour.io/gateway-controller
```

One per implementation in the cluster. Usually pre-installed by the platform team. You'll see `cilium`, `contour`, `nginx`, `gke-l7-global`, ...

Gateway

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata: { name: public }
spec:
  gatewayClassName: contour
  listeners:
    - name: http
      protocol: HTTP
      port: 80
      allowedRoutes:
        namespaces: { from: All }
    - name: https
      protocol: HTTPS
      port: 443
      tls:
        certificateRefs:
          - { kind: Secret, name: gw-tls }
      allowedRoutes:
        namespaces:
          from: Selector
          selector: { matchLabels: { gw-attach: yes } }
```

HTTPRoute

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: app
  namespace: prod
spec:
  parentRefs:
    - { name: public, namespace: infra }      # attach to a Gateway
  hostnames: [app.example.com]
  rules:
    - matches:
        - path: { type: PathPrefix, value: /api }
      backendRefs:
        - { name: api, port: 80, weight: 80 }
        - { name: api-canary, port: 80, weight: 20 }  # traffic split
    - matches:
        - path: { type: PathPrefix, value: / }
      backendRefs:
        - { name: web, port: 80 }
```

Defines **where the traffic goes**. Note: built-in traffic splitting via weights, no annotations needed.

Cross-namespace attachment

Gateways live in one namespace, HTTPRoutes can attach from others — controlled by **ReferenceGrant**:

```
apiVersion: gateway.networking.k8s.io/v1beta1
kind: ReferenceGrant
metadata: { name: route-to-svc, namespace: prod }
spec:
  from:
    - { group: gateway.networking.k8s.io, kind: HTTPRoute, namespace: prod }
  to:
    - { group: "", kind: Service }
```

This is how **ParentRefs** (route → Gateway) and **BackendRefs** (route → Service in another namespace) get explicit permission. Security-by-default.

Gateway API vs Ingress — when to use which

Need	Use
Simple HTTP routing, you control all the YAML	Ingress is fine
TCP/UDP/gRPC	Gateway API
Traffic splitting / canary	Gateway API (no annotation soup)
Cross-team / cross-namespace boundaries	Gateway API (ReferenceGrant)
Header rewriting, redirects in spec	Gateway API
Already heavily invested in ingress-nginx	Ingress, migrate gradually

On the CKA, you should be able to write an **HTTPRoute** that splits traffic by host or path. Ingress still appears more often.

Lab 3 (part 2) — Ingress

→ `trainees/day2/labs/lab3-services-ingress.md` (Section 4+)

20 min. Install ingress-nginx on kind (`extraPortMappings` already in the kind config). Create an Ingress for `web.local`. Add `127.0.0.1 web.local` to `/etc/hosts`. `curl` it.

Lab 3b — Gateway API mini

→ `trainees/day2/labs/lab3b-gateway-api.md`

25 min. Install Gateway API CRDs + Contour. Create a Gateway + HTTPRoute. Try a 80/20 traffic split between two Deployments. Observe with `curl` in a loop.

NetworkPolicy — the model

By default, every pod can talk to every other pod across all namespaces (pod-to-pod fully open).

NetworkPolicy is the pod-level firewall.

- Cluster needs a CNI that **enforces** NetworkPolicy. Calico , Cilium . **kindnet does NOT enforce.**
- Two policy types: **Ingress** (who can connect TO me) and **Egress** (who I can connect TO)
- Policies are **additive** — multiple policies that select the same pod **OR** their rules together

Default-deny — the canonical YAML

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: prod
spec:
  podSelector: {} # ALL pods in this namespace
  policyTypes: [Ingress, Egress]
  # no ingress: or egress: block → deny everything
```

podSelector: {} = match all pods. Empty **policyTypes** deny lists = block all. **This must be memorized.**

Add allow-rules in additional policies on top.

Allow specific ingress

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend
  namespace: prod
spec:
  podSelector:
    matchLabels: { role: backend }
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector:
            matchLabels: { role: frontend }
      ports:
        - { protocol: TCP, port: 8080 }
```

"Backend pods accept TCP/8080 from frontend pods (same namespace)."

The AND vs OR trap

```
# AND — same list item
ingress:
  - from:
    - namespaceSelector: { matchLabels: { team: a } }
      podSelector:       { matchLabels: { app: web } }    # both must match

# OR — separate list items
ingress:
  - from:
    - namespaceSelector: { matchLabels: { team: a } }    # any pod in those namespaces
    - podSelector:       { matchLabels: { app: web } }   # any web pod in any namespace
```

The single hyphen vs separate hyphens matters. This is *the* exam trap.

Egress to DNS — a common pattern

```
egress:
- to:
  namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: kube-system } }
  podSelector:       { matchLabels: { k8s-app: kube-dns } }
ports:
- { protocol: UDP, port: 53 }
- { protocol: TCP, port: 53 }
```

Default-deny egress and you've also broken DNS. Always pair default-deny with an allow-DNS rule, or trainees will spend 20 min wondering why nothing resolves.

`kubernetes.io/metadata.name` is an automatic label put on every Namespace by kubelet — handy for selecting `kube-system` without re-labeling.

ipBlock — by CIDR

```
ingress:
- from:
  - ipBlock:
      cidr: 10.0.0.0/16
      except:
        - 10.0.5.0/24
  ports: [{ protocol: TCP, port: 443 }]
```

Use when source is **not** a pod (external IPs, on-prem networks). NetworkPolicy is one of the few places you can express L3 source filters cleanly.

Default behaviors to internalize

If you have...	Effect
No NetworkPolicy on a pod	All ingress + egress allowed (the open default)
A policy selecting the pod, policyTypes includes <code>Ingress</code> , no <code>ingress:</code> block	All ingress denied
A policy selecting the pod, policyTypes only <code>Ingress</code>	Egress still unrestricted
Multiple policies select the pod	Their allow rules OR together

There is **no deny rule**. NetworkPolicy is allow-list-only. "Deny" is the absence of allow.

NetworkPolicy debugging

1. `k describe networkpolicy -n <ns>` — read every active policy
2. Trace from sender: open a shell in the source pod, `curl` the destination. Get refused or timed out?
Timeout usually = NetworkPolicy.
3. Check the **CNI's** logs/observability:
 - Calico: `calicoctl get networkpolicy --all-namespaces`; `kubectl logs -n calico-system calico-node-*`
 - Cilium: `cilium connectivity test`; `cilium hubble observe` (powerful)
4. If on kind: check you swapped from kindnet to Calico/Cilium. *kindnet ignores policies.*

Lab 4 — NetworkPolicy

→ `trainees/day2/labs/lab4-networkpolicy.md`

45 min. Lab is on the Calico-enabled cluster — trainer will reset before the lab if needed.

- Apply default-deny on a namespace
- Verify pods can't reach each other or out
- Add allow-frontend-to-backend
- Add allow-egress-DNS so resolution works
- Add cross-namespace allow via namespaceSelector

Day 2 wrap-up

You can now:

- Pick the right Service type and wire endpoints
- Debug DNS — search domains, ndots, CoreDNS Corefile
- Write Ingress with IngressClass + TLS
- Write Gateway + HTTPRoute, including traffic splits
- Lock down a namespace with NetworkPolicy and unbreak DNS afterwards

Tonight:

- 1 killercoda **networking** scenario
- Read `kubernetes.io/docs/concepts/services-networking/network-policies/` end-to-end
- Skim `gateway-api.sigs.k8s.io` "Getting Started" page

Tomorrow: Scheduling, Storage, RBAC, Helm, Kustomize, HPA — busiest day.